

Resumen NLP — Unidades 1 y 2

2026-09-19 · @Someone

1. Extracción de texto: definición y fuentes (Unidad 1)

🔑 **Extracción de texto (Text Extraction):** proceso de recuperar texto plano desde formatos de almacenamiento (PDF, HTML, audio transcrito, escaneos). No confundir con **extracción de información**, que extrae entidades y relaciones desde texto ya extraído. Este concepto es la base de todo el pipeline de NLP.

Fuentes típicas de texto: documentos de texto, páginas web, redes sociales, correo electrónico, bases de datos, transcripciones, literatura. Cada fuente requiere técnicas y librerías distintas.

Formato TXT: texto plano, sin estilos. Requiere especificar el *encoding* al abrir el archivo (`utf-8` , `iso-8859-1` , `latin-1`). Un *encoding* incorrecto lanza `UnicodeDecodeError` . Distintos encodings generan distinta cantidad de bytes para el mismo texto.

Formato HTML: usa etiquetas (`<p>` , `<h1>` - `<h6>` , `<strong>` , `<em>`) para estructurar contenido. Se extrae texto plano con la librería `BeautifulSoup` combinada con `lxml` . El método `.text` remueve las etiquetas y conserva solo el contenido.

Formato Markdown (.md): sintaxis liviana para dar formato. Se convierte a HTML o texto plano con las librerías `markdown` o `mistune` . Para texto plano se combina `markdown` + `html2text` .

Relación con otros temas: estos tres formatos son la base sobre la que luego se aplican limpieza, tokenización y segmentación (secciones 5, 6 y 7).

2. Formato PDF, texto en imágenes (OCR) y DOCX (Unidad 1)

🔑 **Formato PDF:** guarda instrucciones de dibujo, no párrafos. Estructura: cabecera, cuerpo (objetos), tabla de referencias cruzadas, trailer, EOF. Extraer texto plano puede perder columnas y tablas.

Librerías para extraer texto de PDF: `PyPDF2` (básica, falla con texto-imagen), `pdfplumber` (buena para tablas), `pypdf` (sucesor de `PyPDF2`), `PyMuPDF` (rápida). `PyPDF2` también extrae metadatos (autor, título, cantidad de páginas).

PDF escaneado (texto como imagen): se convierte cada página a imagen con `pdf2image` (requiere `poppler-utils`). Luego se aplica OCR sobre esas imágenes.

🔑 **OCR (Optical Character Recognition):** convierte una imagen en texto. `pytesseract` es un wrapper de Tesseract OCR y soporta varios idiomas (`lang="eng+spa+fra"`). `EasyOCR` además devuelve la ubicación (coordenadas) de cada bloque de texto. El OCR no es 100% preciso; el preprocesamiento de imágenes (binarización, suavizado) mejora el resultado.

Formato DOCX: basado en XML. Se lee con la librería `python-docx` . El atributo `doc.paragraphs` entrega el texto de cada párrafo.

Conclusión de la sección: la elección de librería depende del tipo de documento. Un PDF con texto embebido no necesita OCR; un PDF escaneado sí lo necesita.

3. Conversores de documentos y extracción multimedia (Unidad 1)

Conversores de documentos: reconstruyen la estructura del documento (títulos, tablas, orden de lectura) y la vuelcan a Markdown. Necesarios porque `.extract_text()` pierde estructura.

Herramienta	Rol	Cuándo usarla
MarkItDown	Wrapper liviano a Markdown	Prototipos, muchos formatos
Docling	Parser estructurado local	Tablas, reading order
Marker	Calidad en PDF difícil	Papers, fórmulas, con GPU
MinerU	Parser de documentos complejos	Scans, OCR multilenguaje

Texto desde audio: `SpeechRecognition` transcribe archivos `.wav` con el motor de Google (`recognize_google`). Un video requiere extraer antes la pista de audio con `ffmpeg` .

🔑 **Modelo Whisper:** modelo de reconocimiento de voz de propósito general, tipo secuencia-a-secuencia (Transformer). Es multitarea: transcribe, traduce e identifica idiomas.

Transcripciones de YouTube: la librería `youtube-transcript-api` devuelve texto, marca de tiempo (`start`) y duración por fragmento.

Texto desde Wikipedia: la librería `wikipediaapi` obtiene resumen (`page.summary`) y contenido completo (`page.text`) de un artículo.

Texto desde bases de datos: `sqlite3` , `Psycopg2` (PostgreSQL), `PyMySQL` y `SQLAlchemy` permiten ejecutar consultas SQL y extraer texto almacenado.

Texto desde formatos tabulares: `pandas` lee Excel (`read_excel`), CSV (`read_csv`) y Parquet; se extrae una columna como serie de texto.

Texto desde JSON: la librería `json` lee archivos locales; `requests` + `json` descargan y parsean JSON desde la web.

4. Web Scraping y parseo de texto (Unidad 1)

 **Web Scraping:** técnica para extraer información de sitios web. Convierte HTML en formato estructurado (CSV, JSON, XML). Debe respetar `robots.txt` y los Términos de Servicio del sitio.

Librerías de scraping: `BeautifulSoup` (parseo estático con `requests`), `scrapy` , `selenium` y `playwright` (renderizan JavaScript, permiten capturas y navegación).

Sesiones y login: el objeto `requests.Session()` persiste cookies entre solicitudes. Permite hacer login (POST) y luego acceder a páginas protegidas (GET) manteniendo la sesión.

Parseo de texto (parsing): analiza una cadena para extraer, interpretar o estructurar su contenido. Convierte datos no estructurados en formatos utilizables (listas, diccionarios).

Herramientas de parseo: el módulo `re` (expresiones regulares) extrae patrones como emails o teléfonos. La librería `parse` extrae valores desde un patrón con placeholders (`"{}-{}-{}"`).

Relación con temas previos: el scraping combina extracción (sección 1) con parseo, y su salida requiere luego limpieza (sección 5).

5. Limpieza y normalización de texto (Unidad 1)

 **Limpieza y normalización:** conjunto de técnicas que estandarizan el texto antes de procesarlo. Incluye conversión a minúsculas, eliminación de puntuación, acentos y stopwords.

Conversión a minúsculas: se aplica con `.lower()` de Python o `.str.lower()` de Pandas. Evita que "Python" y "python" se traten como palabras distintas.

Eliminación de puntuación: se usa `re.sub(r'[^w\s]', '', texto)` . No siempre conviene: los modelos tipo LLM suelen trabajar con la puntuación original.

Eliminación de acentos: la función `unicodedata.normalize('NFKD', texto)` descompone cada carácter y filtra los combinantes (acentos). Homogeneiza texto de distintas fuentes.

🔑 **Eliminación de stopwords:** quita palabras muy frecuentes sin significado propio ("el", "la", "de"). Reduce dimensionalidad y ruido. Cuidado: negaciones ("no") e intensificadores ("muy") pueden ser relevantes en análisis de sentimientos.

Estandarización de texto: reemplaza abreviaturas por su forma completa usando un diccionario de búsqueda (`lookup_dict`), combinada con limpieza de puntuación.

Corrección ortográfica: debe aplicarse después de expandir abreviaturas. Librerías: `TextBlob` (mejor en inglés), `pyspellchecker` y `autocorrect` (soportan español).

Relación con otros temas: esta limpieza precede a la tokenización (sección 6) y a la codificación vectorial (sección 8).

6. Tokenización, stemming, lematización y emojis (Unidad 1)

🔑 **Tokenización:** divide el texto en unidades mínimas (palabras u oraciones). Es un paso obligatorio antes de cualquier análisis. Librerías: `NLTK` (`word_tokenize`), `spaCy` , `TextBlob` , `stanza` y `pysbd` (soporta 22 idiomas).

Elección de la unidad: tokenizar define qué cuenta como evidencia para el modelo. Afecta el manejo de puntuación, emojis, tildes y contracciones.

🔑 **Derivación (stemming):** reduce una palabra a su raíz, sin garantizar que sea una palabra real. Usa reglas heurísticas (algoritmo Snowball). Es rápido pero puede aplanar matices ("bueno", "buenísimo" → "buen").

🔑 **Lematización:** reduce una palabra a su forma base válida en el idioma, usando contexto y análisis morfológico. Más lenta pero más precisa que el stemming. En español se usa `spaCy` con el modelo `es_core_news_sm` .

Diferencia clave stemming vs. lematización: el stemming no requiere diccionario; la lematización sí, y por eso conserva vocabulario real.

Tratamiento de emojis: tres estrategias: eliminarlos, convertirlos a texto (`emoji.demojize` , `demoji`) o tratarlos como palabras. Las librerías `emot` y `demoji` identifican emojis y emoticonos con su significado.

7. Analítica de texto y segmentación (Unidad 1)

Analítica de texto: incluye frecuencia de palabras (`FreqDist` de `NLTK`), estructura del texto, n-gramas, co-ocurrencia y correlación de palabras.

N-gramas: secuencia contigua de n palabras. Útiles para predecir la siguiente palabra, corrección ortográfica y traducción automática. Se generan con `nltk.util.ngrams` .

Co-ocurrencia de palabras: frecuencia con la que dos palabras aparecen juntas, dentro de una ventana de palabras o de un límite estructural (oración, párrafo). Revela relaciones semánticas.

Correlación de palabras: mide si la presencia de un término se asocia estadísticamente con otro, considerando también su ausencia. Usa el coeficiente de Pearson. Es más estricta que la co-ocurrencia: dos palabras pueden co-ocurrir por azar sin estar correlacionadas.

🔑 **Segmentación de texto (chunking):** divide el texto en unidades manejables ("chunks") para búsqueda semántica y sistemas RAG. Existen cuatro enfoques principales:

1. **Tamaño fijo:** divide por cantidad de tokens, con solapamiento opcional (`CharacterTextSplitter` de langchain).
2. **Consciente del contenido:** divide por oraciones, usando `nltk`, `spaCy`, `stanza` o `pysbd`.
3. **Recursiva:** aplica separadores en cascada (`\n\n`, `\n`, espacio) hasta lograr el tamaño buscado (`RecursiveCharacterTextSplitter`). El parámetro `chunk_overlap` preserva contexto entre fragmentos.
4. **Especializada:** respeta la estructura de Markdown o LaTeX (`MarkdownTextSplitter`, `LatexTextSplitter`).

🔑 **Segmentación semántica:** técnica avanzada que agrupa oraciones por significado, usando embeddings, en vez de reglas de tamaño (`SemanticChunker` de `chonkie`). Anticipa el tema de la Unidad 2.

8. Codificación de texto en vectores (Unidad 2)

🔑 **Codificación vectorial:** convierte texto en números para que los algoritmos de aprendizaje automático lo procesen. Es el puente entre NLP y Machine Learning.

One-hot encoding: representa cada palabra como un vector de tamaño n (vocabulario), con un 1 en su posición y 0 en el resto. Simple, pero genera alta dimensionalidad y no captura similitud semántica: todas las palabras son ortogonales entre sí. Se implementa con `sklearn.OneHotEncoder` o `pd.get_dummies`.

🔑 **Count Vectorizer:** cuenta cuántas veces aparece cada palabra en cada documento. No respeta el orden de las palabras (bolsa de palabras). Da igual peso a palabras frecuentes poco informativas ("el", "un"). Se implementa con `sklearn.CountVectorizer`.

🔑 **TF-IDF (Term Frequency-Inverse Document Frequency):** pondera cada palabra según su frecuencia en el documento (TF) y su rareza en el corpus (IDF). Sube el valor de palabras frecuentes en un documento; lo baja si la palabra es común en todo el corpus.

Fórmula: $TF\text{-}IDF = TF \times IDF$. Se implementa con `sklearn.TfidfVectorizer`.

Vectorización de hash (Hashing Vectorizer): convierte cada palabra en un índice numérico vía una función hash (`sklearn.HashingVectorizer`). No requiere mantener vocabulario, útil con vocabularios muy grandes. Desventaja: no permite mapear el vector de vuelta a las palabras originales; puede haber colisiones de hash.

🔑 **Limitación común a estos cuatro métodos:** ninguno captura la semántica ni la relación entre palabras. Tratan cada palabra como una entidad aislada. Esto motiva el uso de word embeddings (sección 9).

9. Word embeddings: Word2Vec, FastText y similitud de coseno (Unidad 2)

🔑 **Word Embeddings:** representan palabras como vectores densos de baja dimensión, aprendidos a partir de datos. Palabras con contextos similares obtienen vectores cercanos. Capturan semántica y relaciones ("rey" - "hombre" + "mujer" $\approx$ "reina").

Diferencia con one-hot: one-hot es disperso, de alta dimensión y rígido. Los embeddings son densos, de baja dimensión y aprendidos, no codificados a mano.

🔑 **Word2Vec:** algoritmo de 2013, pionero en embeddings densos. Tiene dos arquitecturas de entrenamiento:

- **CBOW (Continuous Bag of Words):** predice la palabra central a partir de su contexto.
- **Skip-gram:** predice las palabras de contexto a partir de la palabra central. No distingue orden ni dirección de las palabras.

Se entrena e implementa con `gensim.models.Word2Vec` (parámetro `sg=0` para CBOW, `sg=1` para Skip-gram). Se pueden cargar modelos preentrenados con `KeyedVectors.load_word2vec_format`.

🔑 **FastText:** extiende Word2Vec aprendiendo vectores de n-gramas de caracteres, no de palabras completas. Ventajas: captura morfología (gato, gatos, gatito comparten n-gramas) y resuelve palabras fuera de vocabulario (OOV). Sigue siendo un embedding estático: un único vector por palabra, sin distinguir sentidos según el contexto.

🔑 **Similitud de coseno:** mide el ángulo entre dos vectores. Fórmula: $\cos(\theta) = (A \cdot B) / (\|A\| \|B\|)$. Valores: 1 = idénticos, 0 = ortogonales, -1 = opuestos. Se prefiere sobre la distancia euclidiana por ser invariante a la magnitud del vector y más eficiente en alta dimensión. Se calcula con `sklearn.metrics.pairwise.cosine_similarity`.

10. Embeddings contextuales, de oraciones y MTEB (Unidad 2)

Tipos de embeddings según el contexto:

- **Contexto-independiente (clásicos):** un vector único por palabra, sin importar la oración. Ejemplos: Word2Vec, GloVe, FastText.
- **Contexto-dependiente:** generan distinto vector para la misma palabra según su uso. Basados en RNN (CoVe, ELMo) o en Transformers.

🔑 **BERT:** modelo basado en Transformers, contextual. El vector de "banco" cambia según la oración (dinero vs. plaza). Marca el corte respecto de Word2Vec/FastText, que son estáticos.

Línea temporal de hitos (resumen): Word2Vec (2013) → GloVe (2014) → Doc2Vec (2014) → FastText (2016) → InferSent (2017) → ELMo (2018) → BERT (2018) → USE (2018) → Sentence-BERT (2019) → MTEB (2022) → EmbeddingGemma (2025). GloVe no aporta una idea nueva frente a Word2Vec, solo otro entrenamiento.

Visualización de embeddings: al tener muchas dimensiones, se reducen a 2D o 3D con **t-SNE** (preserva relaciones de vecindad) o **PCA** (captura la mayor varianza).

🔑 **Embeddings en oraciones (sentence embeddings):** representan una oración completa como un vector. El promedio de word embeddings es una técnica simple pero limitada: pierde información y el orden no afecta el resultado.

🔑 **Sentence-BERT (S-BERT):** modificación de BERT que usa redes siamesas y tripletas para generar embeddings de oraciones comparables por similitud de coseno. Resuelve la limitación de BERT (cross-encoder, computacionalmente costoso) para búsqueda semántica. Se implementa con la librería `sentence-transformers`.

🔑 **MTEB (Massive Text Embedding Benchmark):** evalúa modelos de embeddings en 8 tareas y 112 idiomas (clasificación, clustering, retrieval, similitud semántica, entre otras). Sirve para elegir un modelo preentrenado según idioma y tamaño, sin necesidad de reentrenar.

Relación general de la Unidad 2: one-hot, Count Vectorizer, TF-IDF y Hashing no capturan semántica (sección 8). Word2Vec y FastText sí, pero son estáticos (sección 9). BERT y Sentence-BERT agregan contexto (esta sección).